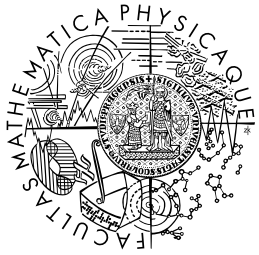

FACULTY
OF MATHEMATICS
AND PHYSICS
Charles University

BACHELOR THESIS

Kinan Al-Falakh

Automating Benchmark Analysis Through Machine Learning

Department of Distributed and Dependable Systems

Supervisor of the bachelor thesis: Mgr. Vojtěch Horký, Ph.D.
Study programme: Computer Science

Prague 2026

I declare that I carried out this bachelor thesis on my own, and only with the cited sources, literature and other professional sources. I understand that my work relates to the rights and obligations under the Act No. 121/2000 Sb., the Copyright Act, as amended, in particular the fact that the Charles University has the right to conclude a license agreement on the use of this work as a school work pursuant to Section 60 subsection 1 of the Copyright Act.

In ..... date ....

Author's signature

Dedication.

Title: Automating Benchmark Analysis Through Machine Learning

Author: Kinan Al-Falakh

Department: Department of Distributed and Dependable Systems

Supervisor: Mgr. Vojtěch Horký, Ph.D., Department of Distributed and Dependable Systems

Abstract: Regular software performance benchmarking is used to prevent silent performance deterioration, yet interpreting the results remains challenging due to non-deterministic program behavior. This thesis employs machine learning, specifically contrastive time series representation learning via TS2Vec, to automate the detection of performance changes in GraalVM benchmark suites. Raw iteration times are normalized using a log-transform followed by robust IQR scaling to handle multiplicative noise from garbage collection and compilation activity. A single dilated convolutional encoder, trained configuration-independently with hierarchical temporal contrastive loss, maps variable-length benchmark runs to fixed-size embedding vectors. Performance regressions are then detected as anomalous transitions in embedding space using cosine distance between consecutive versions. The method is evaluated on GraalVM benchmark data collected from 2016 to 2022 across multiple domain-specific configurations.

Keywords: machine learning, software benchmarking, automated analysis, contrastive learning, time series

Contents

Introduction **7**

1 Benchmarking and the GraalVM Dataset **8**
1.1 Software Performance Benchmarking 8
1.2 JVM Execution Characteristics 8
1.2.1 JIT Compilation 8
1.2.2 Garbage Collection 8
1.2.3 Typical Benchmark Execution Profile 8
1.3 The GraalVM Compiler Benchmark Results Dataset 8
1.3.1 Data Source and Collection Context 8
1.3.2 Dataset Structure 8
1.3.3 Measurement Metadata 8
1.4 Configuration as the Unit of Analysis 8
1.4.1 Defining a Configuration 8
1.4.2 Metadata Resolution and Filtering 8
1.5 Version Deduplication 8

2 Preprocessing **9**
2.1 Steady-State Detection 9
2.1.1 The JVM Warmup Problem 9
2.1.2 Steady-State Detector Algorithm 9
2.2 Normalization 9
2.2.1 Noise in Benchmark Measurements 9
2.2.2 Why Min-Max Normalization Fails 9
2.2.3 Why Z-Score Normalization Fails 9
2.2.4 Log-Transform: Converting Multiplicative Noise to Additive 9
2.2.5 Robust Scaling with Median and IQR 9
2.2.6 Per-Configuration Application 9
2.3 Padding and Batching Variable-Length Sequences 9

3 Related Work **10**
3.1 Performance Regression Detection 10
3.1.1 Statistical Approaches 10
3.1.2 Machine Learning Approaches 10
3.2 Alternative Approaches 10
3.2.1 Statistical Feature Extraction 10
3.2.2 LSTM Autoencoders 10
3.3 TS2Vec and Contrastive Learning 10
3.4 Gap: Time Series Representations for Benchmark Analysis 10

4 TS2Vec and Contrastive Learning **11**

5 Anomaly Detection Pipeline and Reporting **12**

Conclusion **13**

List of Figures **14**

List of Tables **15**

List of Abbreviations **16**

A Attachments **17**
A.1 First Attachment 17

Introduction

Regular software performance benchmarking is employed to prevent silent performance deterioration of software products. While automating the execution of benchmarks is straightforward, the real challenge lies in interpreting the results: software rarely behaves in a fully deterministic manner, and sophisticated methods must be used to detect genuine performance changes amid noise.

...

1 Benchmarking and the GraalVM Dataset

1.1 Software Performance Benchmarking

1.2 JVM Execution Characteristics

1.2.1 JIT Compilation

1.2.2 Garbage Collection

1.2.3 Typical Benchmark Execution Profile

1.3 The GraalVM Compiler Benchmark Results Dataset

1.3.1 Data Source and Collection Context

1.3.2 Dataset Structure

1.3.3 Measurement Metadata

1.4 Configuration as the Unit of Analysis

1.4.1 Defining a Configuration

1.4.2 Metadata Resolution and Filtering

1.5 Version Deduplication

2 Preprocessing

2.1 Steady-State Detection

2.1.1 The JVM Warmup Problem

2.1.2 Steady-State Detector Algorithm

2.2 Normalization

2.2.1 Noise in Benchmark Measurements

Multiplicative Nature of JVM Noise

Garbage Collection Spikes

OS Scheduling Jitter

2.2.2 Why Min-Max Normalization Fails

2.2.3 Why Z-Score Normalization Fails

2.2.4 Log-Transform: Converting Multiplicative Noise to Additive

2.2.5 Robust Scaling with Median and IQR

2.2.6 Per-Configuration Application

2.3 Padding and Batching Variable-Length Sequences

3 Related Work

3.1 Performance Regression Detection

3.1.1 Statistical Approaches

3.1.2 Machine Learning Approaches

3.2 Alternative Approaches

3.2.1 Statistical Feature Extraction

3.2.2 LSTM Autoencoders

3.3 TS2Vec and Contrastive Learning

3.4 Gap: Time Series Representations for Benchmark Analysis

4 TS2Vec and Contrastive Learning

5 Anomaly Detection Pipeline and Reporting

Conclusion

List of Figures

List of Tables

List of Abbreviations

A Attachments

A.1 First Attachment